AI時代的多工與另類心流

李智樺

AI 不一定先取代「人」，而是先取代「工作中的某些活動」

專案開始之初，首重看見全貌

一旦；當你把眼光投注在哪一個要項的時候，
實際上你就只看到那一部分，

你的思緒將被那一部分的 內容所牽動，很難再看見其他的事

…

所以我們要退後一步，

不！有時要退後很多步，才能比較清晰地看見全貌。

- 為何會多工？ Vibe coding 對心流的影響？

心流

AI 時代的工程師

從實作解法，到形成整體理解

規劃與分析，也能成為心流的場域

實作中的心流

讓解法逐步成形



實作技能 ↔ 技術挑戰

修改程式、測試與除錯

回饋： 功能逐步運作

理解中的心流

讓整個計畫逐步清晰



理解能力 ↔ 問題複雜度

提問、建模、推演與驗證

回饋：關係逐步清楚，下一步可驗證

讓心流中的愉悅與實際學習相連接

理解形成的心流

「我逐步看懂整個計畫，也知道如何驗證下一步。」

概念分析：心流是否出現，仍取決於個人與情境

透過對「行動的理解」進入心流

四種活動，共同深化對工作的理解

分析

發現問題背後的結構



規劃

推演哪些行動能帶來進展



協調

根據理解，安排人與工作的關係



實作

將理解具體化，
接受現實的檢驗



形成理解，採取行動，再用結果修正理解。

多工

工作重組的副作用：多工

1 工程師角色轉變



工程師開始從「**執行者**」變成「**工作協調者**」。

2 新的工作情境



早上，工程師讓 AI 處理 **Task A** 的程式修改；
等待 AI 執行測試的同時，另一個 AI 正在分析 **Task B** 的架構問題；Production 又出現 **Task C** 的 Bug，AI 已經開始分析 Log 了。

3 三個 AI agent 同時工作著

- A 程式修改 - 測試
- B 分析架構問題
- C Bug - 分析 Log

- **AI Task A** → 程式修改 - 測試 (- - → 偷懶、忽略 ...)
- **AI Task B** → 分析架構問題
- **AI Task C** → Bug - 分析 Log



同時進行的三條 AI 工作流



4 面臨的挑戰



工程師不可能同時深入理解三個架構問題、
同時做三個重要決策。

5 人類的工作仍然比較接近



「**Human Focus：A → B → C → A**」

大綱

01 工作重組

善用 AI，始於好的工作重組



必然性多工

02 動態專注

在多工中重新體驗心流



成就心流

03 Code review 促成理解

工程師的產出：理解



促成理解



01 工作重組
AI 拆工作，人重新分工

02 動態專注
多脈絡，單一專注

03 理解驗證
AI 產出 Code，人產出理解

從「自己做」→「與 AI 協作」→「對結果負責」

Code Review

AI 時代的工程師

理解的五個層次

從知道改動，到預測行為

理解逐步深化

第五層 | 我可以預測修改後的行為？



第四層 | 我知道失敗時去哪裡找證據？



第三層 | 我知道哪裡可能失敗？



第二層 | 我可以解釋運作的結構？



第一層 | 我知道 AI 改了甚麼？



理解愈深，愈能解釋、查證與預測。

用好 AI，先重新設計工作

拆開工作，才能重新安排人與 AI 的協作

拆開 × 重組

小增量、多迭代

看清工作單元

- 需求 → 成果
- 完成 → 邊界
- 假設 → 未知
- 數據 → 證據

每個單元都有輸入、產出與完成條件

重新安排

接成人機協作流程

人：確認意圖 → AI：產出方案 → 人+AI：驗證修正

迭代回饋

依風險、可驗證性與脈絡需求分工

Human in the Loop

調整順序 | 明確交接 | 縮短回饋 | 刪除多餘工作

把 SOP 寫在對的層次

可重用的協作資源

Prompt 框架｜團隊知識庫｜驗收範例

支援工作單元，讓協作可以重複使用

有層次的驗證與理解

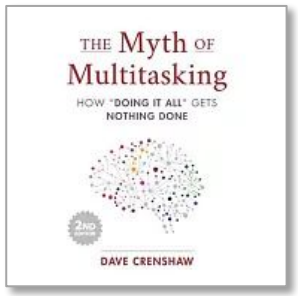
有依據的品質審查

誰負責？查什麼？憑什麼放行？

以 Code Review 理解五層與證據確認影響

拆得清楚，接得順暢，驗得有據。

提問



多任務的神話

The Myth of Multitasking



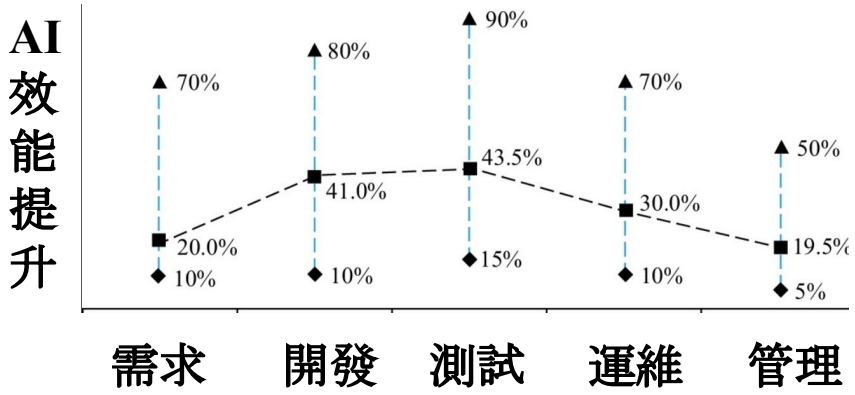
戴夫·克倫肖

- Dave Crenshaw

2021/02/09

人腦無法同時處理兩個需要高度注意力的主動任務 (active tasks)。
我們只是快速在不同任務之間做切換，每次切換都會產生「切換成本」
switch cost：

- **處理時間變長** (可能多花 **20-40%** 時間)
- **錯誤增加**
- **壓力上升**
- **工作品質下降**
- **人際關係受損** (因爲無法真正專注聆聽對方在說甚麼)



兩種不同的「多工」

- **Switchtasking(切換任務)：有害**，應該盡量避免 (例如一邊開會一邊回 email)。
- **Background Tasking: 無害或甚至有幫助** (例:跑步時聽有聲書、機械性工作時聽音樂)。

備一

任務被中斷程式時的損失

- 主要損失在 **運算時間**、**記憶體內的未儲存資料**，以及工程師的**專注力與開發情境**。
- **短期與進行中損失：未儲存的進度 (Unsaved Data)、執行時間 (Compute Time)**
- **中期與系統層面損失狀態不一致 (Inconsistent State)**
- **人為因素損失心智模型 (Mental Model)**：工程師大腦中對於目前程式碼邏輯、變數關係與架構的全盤理解會被打斷，事後需耗費大量的時間重新回憶與除錯。

▶ 為避免此類損失，開發者通常會導入**檢查點機制 (Checkpoints)**、**自動儲存**、**日誌記錄 (Logging)** 或**事務處理 (Transactions)** 來確保任務即使中斷也能安全恢復。

備二

多工的壞處

	多工的壞處	具體說明	百分比
1	**時間損失**	每次任務切換都需要大腦重新聚焦，累積下來大幅延長總時間	**生產力平均下降 40%任務完成時間可增加 25-100%**
2	**錯誤率上升**	注意力分散導致粗心、遺漏細節	**錯誤率可增加 50% 以上**
3	**專注力與工作品質下降**	大腦無法進入深度工作狀態，創意與解決問題能力減弱	**深度工作能力下降 30-50%**
4	**壓力與疲勞增加**	持續切換造成大腦過度負荷，皮質醇上升	壓力荷爾蒙上升，長期多工者 burnout 風險大幅提高
5	**記憶力與學習效果變差**	注意力殘留(Attention Residue)讓資訊無法有效存入長期記憶	**資訊保留率下降 20-30%**
6	**人際關係受損**	與人互動時不夠專注，容易讓對方感覺不被尊重	溝通品質下降，關係滿意度降低
7	**整體幸福感與生活品質下降**	長期多工導致焦慮、睡眠問題，無法享受當下	**生活滿意度明顯降低**

備三

多工的系統思維

看見工作切換，也要看見背後的系統



表象

工程師不停切換工作

多項任務已開始，卻還沒完成（WIP）

系統結構

等待、插單、依賴，推高在製工作

- 審查與環境等待
- 需求過大或不清
- 跨團隊依賴與插單

效率觀念

忙碌，被當成有效率

- 「人不能閒著」
- 「先開始，就算有進度」

AI 加快開工，若瓶頸不變，待完成工作可能更多。

改善多工，要管理工作流，也要改變效率觀念。

以人為本

多工系統思維

從自然多工，走向動態專注

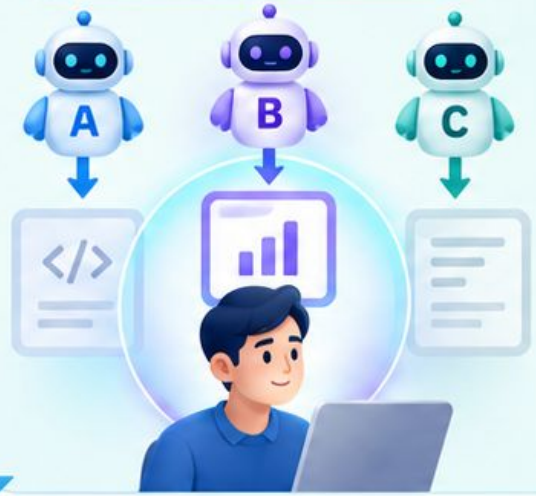
AI 任務並行，人的注意力逐項聚焦

傳統多工



- 多項工作爭奪注意力
- 被動切換，反覆恢復脈絡
- 風險：更多工作壓向更少的人

自然多工



- 委派 AI，任務非同步推進
- 人依序釐清、接手與驗證
- 前提：限制在製工作，保留專注時間

動態專注 Dynamic Focus

多脈絡，單一專注

Multiple Contexts, Single Focus

保存脈絡 → 有意識切換 → 投入當前任務

讓 AI 並行工作，讓人專心完成。

動態專注循環

在多脈絡的世界裡，始終保持單一專注

學會專注
重要的事，深度完成

AI 保存脈絡
快速恢復專注

專注
處理任務
FOCUS

壓縮脈絡
保存 HERE / WHY / NEXT
COMPRESS

AI 保存脈絡

切換
到下一個任務
SWITCH

快速讀取恢復
載入脈絡，繼續專注
RESTORE

恢復脈絡
快速接回
RESUME

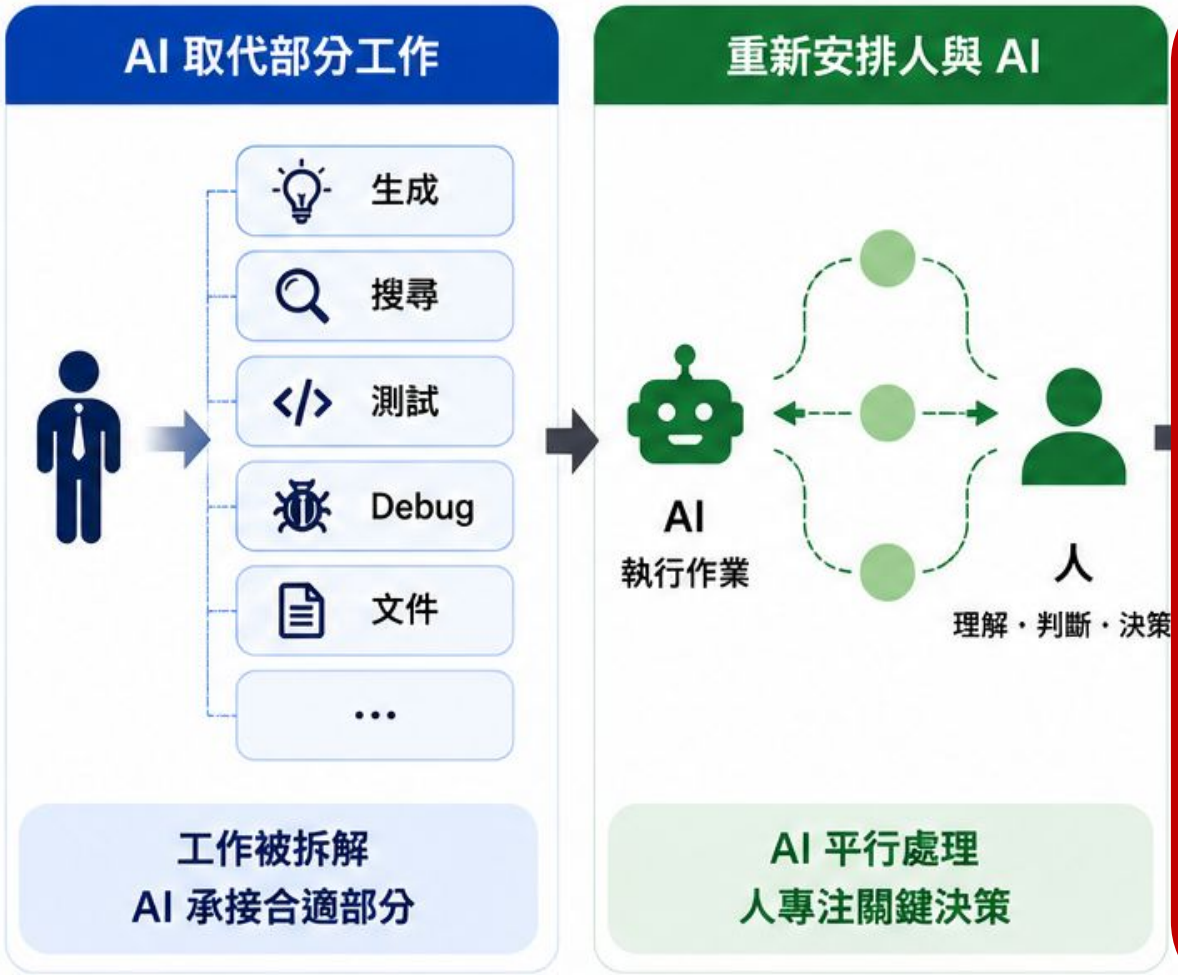
快速恢復專注
回到深度工作狀態
RE-FOCUS

脈絡重啟卡

NOW	我在哪？	做到哪裡・卡在哪裡
THINK	想什麼？	當前假設・思考方向
NEXT	下一步？	回來後要做什麼
WATCH	注意？	風險・別忘記的事

AI 時代的工作演化路徑

1 工作拆解 → 2 工作重組 → 3 多脈絡單一專注 → 4 Dynamic Focus



AI 多工功課表

時間	09:00 - 11:00	11:15 - 13:00	14:00 - 16:00
專注任務	Task A 核心模組開發	Task B 跨團隊評審會議	Task C 緊急修復 (Bug)
任務類型	開發（深度專注）	協作（溝通協調）	緊急處理（快速應對）
主要目標	完成模組 X 功能實作	完成架構評審與 決策共識	定位並修復 生產環境 Bug
課表安排	120 分鐘 深度專注時段	105 分鐘 會議與討論	120 分鐘 專注修復時段
AI 協作重點	• 需求澄清 • 程式碼建議 • Debug 協助	• 會議重點整理 • 決策紀錄 • 行動項目追蹤	• 問題分析 • 修復方案建議 • 影響評估

「 工作拆解;資料蒐集、初步構思、假設 -實驗製作、審核優化」

AI 多工功課表（3 Tasks 範例）

工作 → 封存 → 切換 → 載入 → 恢復 → 工作

時間	09:00 - 11:00	11:15 - 13:00	14:00 - 16:00	16:15 - 17:45	18:30 - 20:00
專注任務	Task A 核心模組開發	Task B 跨團隊評審會議	Task C 緊急修復 (Bug)	Task A 延續開發	Task B 文件整理與追蹤
任務類型	開發（深度專注）	協作（溝通協調）	緊急處理（快速應對）	開發（深度專注）	協作（整理彙整）
主要目標	完成模組 X 功能實作	完成架構評審與 決策共識	定位並修復 生產環境 Bug	延續模組 X 測試與優化	整理會議紀錄與 追蹤行動項目
課表安排	120 分鐘 深度專注時段	105 分鐘 會議與討論	120 分鐘 專注修復時段	90 分鐘 深度專注時段	90 分鐘 整理與追蹤
AI 協作重點	• 需求澄清 • 程式碼建議 • Debug 協助	• 會議重點整理 • 決策紀錄 • 行動項目追蹤	• 問題分析 • 修復方案建議 • 影響評估	• 測試案例建議 • 效能優化建議 • 程式碼 Review	• 文件彙整 • 進度追蹤 • 後續提醒設定

任務清單

- **A** 核心模組開發 — 高優先・高專注
- **B** 跨團隊協作 — 中優先・需溝通
- **C** 緊急修復 (Bug) — 高優先・即時處理

排課原則

- A 為核心，安排最長的深度專注時段
- B 安排在上午與傍晚，利於團隊配合
- C 優先處理，穿插在專注時段之間
- 每次切換都執行「切換儀式」
- AI 協助封存脈絡，降低切換成本

切換儀式

（60~90 秒）

1. 工作 — 專注當前任務
2. 封存 — AI 封存脈絡卡片
3. 切換 — 決定下一個焦點
4. 載入 — 載入新任務脈絡
5. 恢復 — 快速進入狀態
6. 工作 — 開始下一個任務

持續循環

AI 脈絡卡片範本

- 任務目標
- 已完成
- 下一步行動
- 關鍵脈絡（變數、決策、TODO）
- 當前假設 / 思考方向
- 潛在風險與注意事項

AI 時代的多工作業：動態專注循環

人類單點專注，AI 平行執行與記憶脈絡，
反思讓我們越來越會切換。

理解



三個任務的多工範例

	A 核心模組開發	B 架構評審協作	C 緊急 Bug 處理
	深度開發・高專注	溝通協調・中專注	即時回應・高優先

時間	09:00 - 10:30	10:45 - 12:15	13:30 - 15:00	15:15 - 16:30	16:45 - 18:00
AI 平行執行（持續運作）	A AI 正在開發 Login 模組	→ 持續開發與單元測試			
	B AI 準備架構提案與文件	→ 彙整討論意思	→ 更新方案 v2		
	C AI 監控錯誤日誌與分析	→ 嘗試修復方案	→ 等待人類確認		
人類專注（單點）	專注 A：開發 Login 模組程式碼 Review	專注 B：參與架構評審做出決策	專注 C：確認 Bug 根因決定修復方案	專注 A：驗收進度與測試調整需求	專注 B：方案定案追蹤後續工作
切換操作	封存 A → 切換 → 載入 B	封存 B → 切換 → 載入 C	封存 C → 切換 → 載入 A	封存 A → 切換 → 載入 A	封存 A → 切換 → 載入 B

AI 會為每個任務建立「脈絡卡片（Context Card）」

A 範例：Task A 脈絡卡片

目標	完成 Login 模組（支援 OAuth2）
AI 狀態	已完成 Token 驗證，進行 Refresh Token 機制開發
人類決策	採用 Library X；不改動現有 DB Schema
關鍵假設	第三方 OAuth 服務穩定，無需額外快取
待審查	Error Handling 邏輯、單元測試覆蓋率
下次介入點	請離認 Exception 處理策略，並審查測試結果

離開後的變化（Delta Summary）

- 10:30 離開後，AI 完成：
- ✓ 新增 2 個 API
- ✓ 修正 1 處 Null Pointer
- ⚠ 新發現：Refresh Token 過期時間可能太短（風險）
- ? 需人類決定：是否延長 Token 有效時間？

學習

7. 反思（每次切換後 2-3 分鐘）
與 AI 一起檢視本次切換

這次切換有必要嗎？
- 真的需要處理 C 嗎？
- 可以延後或交給 AI 自主完成嗎？

恢復成本高嗎？
- 回來花了多久才進入狀態？
- 哪些脈絡沒被保存好？

下次如何更好？
- 調整切換時機或排程
- 優化 Context Card 欄位
- AI 提升摘要品質

AI 調整下一輪功課表
根據學習結果，
優化排程與切換策略

這套方法帶來的好處
- ☑ 降低切換成本，減少重新理解時間
- ☑ 人類專注在理解、決策與創造
- ☑ AI 平行執行，提升整體產能
- ☑ 透過反思持續進化，越用越順

人類：專注與決策　AI：執行與記憶

經驗

多工轉換的 四個階段

從專注一件事 → 壓縮脈絡 → 快速回來 → 動態多工，讓效能最大化

Level 1 先學會專注 Main Focus

先不要談 AI 多工，先回答：我現在在做什麼？

▶ 每天選一件：Main Focus

- 建立清晰的當下目標
- 減少分心，累積深度產出

Level 2 學會離開 Context Compression

學會好好的「結束一段工作」，回答：我做到哪裡？我現在怎麼想？
下一步是什麼？（用 30~60 秒作回答）

- 把工作脈絡壓縮成精華
- 降低重啟成本，釋放心理負擔

Level 3 學會回來 Context Recovery（縮短 Time to Focus）

回來最重要：回到 Task A 時，不是從頭閱讀。
而是讓 AI 告訴你：① 你上次停在哪裡 ② 原假設是什麼
③ AI 在你離開後完成了什麼 ④ 下一步建議 / 需要你決定什麼
目標不是記憶。而是：縮短 Time to Focus。

- 快速獲得脈絡全景
- 縮短重啟時間，回到高效狀態

Level 4 開始動態多工 開始：A → B → C → A

到了這一層，才開始動態多工：A → B → C → A

▶ 記住：每一次切換都有理由，不是隨機跳來跳去。

- 善用等待時間，AI 並行執行
- 人類動態選擇，保持決策品質

核心價值

保持思考連續性
不論離開多久，都能快速回到狀態

提高交付效能
減少重複理解，專注創造價值

降低壓力與焦慮
AI 幫你記住，讓你安心專注

即使沒有多工需求也有用
每天壓縮與回顧，持續提升效能

不是摘要

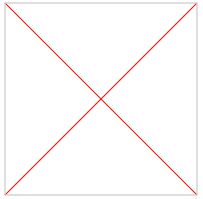
Context Compression 不是摘要

Summary (摘要)是在回答：　　**發生了什麼？**

Memory (記憶)是在回答：　　**我做過什麼？**

但 **Context (脈絡)**是在回答：　　**我為什麼這樣想？**

而 **Recovery (恢復)**則是在回答：**我要如何快速接著想？**



完整的 Context Compression 應該是：

事實壓縮　＋　推理壓縮　＋　意圖壓縮　＋　下一個起點

AI 脈絡重啟卡

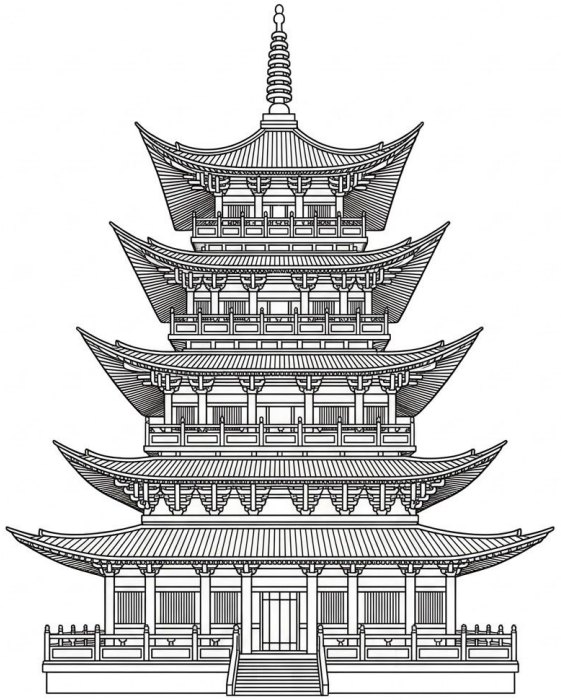
- **NOW** ｜ **我在哪？** - 做到哪裡、卡在哪裡
- **THINK** ｜ **想甚麼？** - 當前假設、思考方向
- **NEXT** ｜ **下一步？** - 回來後要做什麼
- **WATCH** ｜ **注意？** - 風險、別忘記的事

工程師只需要說

- NOW ：DB 連不上已排除
- THINK ：可能是 Connection Pool 出問題
- NEXT ：比較 Production 設定
- WATCH：尖峰流量時才發生

Code Review

- 傳統作法：理解是經由「人眼逐行看程式碼」
- AI 時代的工程師 產出「理解」是 ？



理解五層

第 五 層	我可以預測修改後的行為 ？
第 四 層	我知道失敗時去哪裡找證據?
第 三 層	我知道哪裡可能失敗？
第 二 層	我可以解釋運作的結構 ？
第 一 層	我知道 AI 改了甚麼？

Code Review｜理解五層

理解

工程師自我審視：我理解到哪一層？

AI 時代，工程師的關鍵產出是「理解」



- 第五層 | 真實 | 我可以預測修改後的行為？
- 第四層 | 失敗 | 我知道失敗時去哪裡找證據？
- 第三層 | 行為 | 我知道哪裡可能失敗？
- 第二層 | 結構 | 我可以解釋運作的結構？
- 第一層 | 需求 | 我知道 AI 改了甚麼？

從知道改了什麼，到能預測修改的影響

不看 Code，也能理解

先建立心智模型，再用證據驗證

1 **WHY**
為什麼改？



需求模型

2 **WHAT**
改了什麼？



結構模型

3 **HOW**
怎麼運作？



行為模型

4 **WHAT IF**
如果出錯？



失敗模型

5 **EVIDENCE**
證據在哪？



真實模型



我的心智模型

逐步逼近系統的真實行為



真實的系統

理論

幫理解區分層次

從問題到證據，逐層建立對系統的理解

不只看 Code
而是看見全貌

Level 1
我知道改了什麼？
為什麼要改？解決什麼問題？

Level 2
我可以解釋怎麼運作？
改了哪些東西？範圍是什麼？

Level 3
我知道哪裡可能失敗？
資料怎麼流動？流程如何運作？

Level 4
我知道失敗時去哪裡找證據？
有哪些風險？邊界條件是什麼？

Level 5
我可以預測修改後的行為
怎麼知道它真的如預期運作？



需求模型
理解目標與價值
使用者需求、業務目標
問題背景、成功標準

結構模型
理解組成與關聯
元件、模組、資料結構
依賴關係、介面變更

行為模型
理解動態行為
流程、資料流
時序、狀態轉換

失敗模型
理解異常與風險
錯誤情境、邊界條件
降級策略、復原機制

證據模型
理解實際執行結果
測試、Log、Metrics
Trace、實際觀察結果

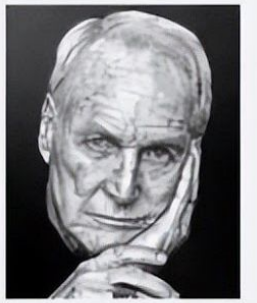
好的理解，是一層一層建立起來的。

From Code to Understanding

依據

布魯姆提問法

布盧姆分類學 Bloom's taxonomy



Benjamin Bloom
本杰明．布魯姆

認知向上的學習過程

發展階段

實作階段

知道階段

Creating 創造: 產生、設計、計畫、製作、發明

Evaluating 評估: 判斷、評價、規劃、批評

Analyzing 分析: 辨別、組織、歸因

Applying 應用: 執行、實行

Understanding 理解: 詮釋、舉例、分類、摘要、推論、比較、解釋

Remembering 記憶: 再認、回憶

問題與回答

10	3	1	3	
【問題】	【分析】	【回答】	【確認】	【完成】
				
【問題】 問題描述	【分析】 加以重點 描述	【回答】 在二分鐘 內做解答	【確認】 提問者確 確認解答	【完成】 最後的 回顧檢視

如果明天出事 ?!

我能說清楚它為什麼會這樣？

STEP 1 | 先不要讓 AI Coding：先把需求改寫成「可驗證的假設」

STEP 2 | **工作拆解**：不要再按照「程式結構」切工作

STEP 4 | AI 產出之後，不要立刻 Code Review

STEP 5 | Code Review 改成「**模型 Review**」

STEP 6 | 從 Code Review 進入「理解 Review」- 回答五件事

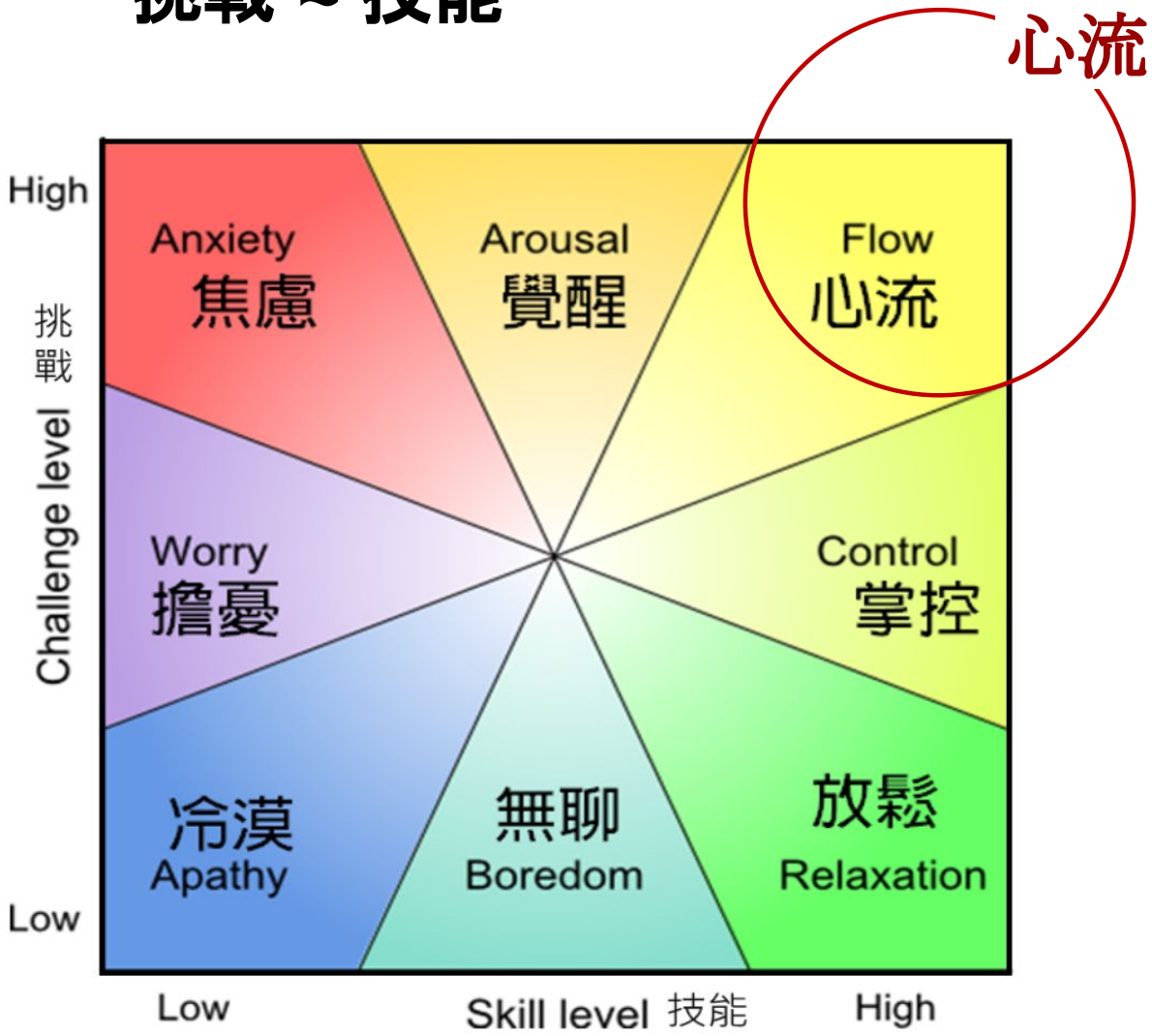
STEP 7 | 把「我覺得懂」變成可觀察的理解程度

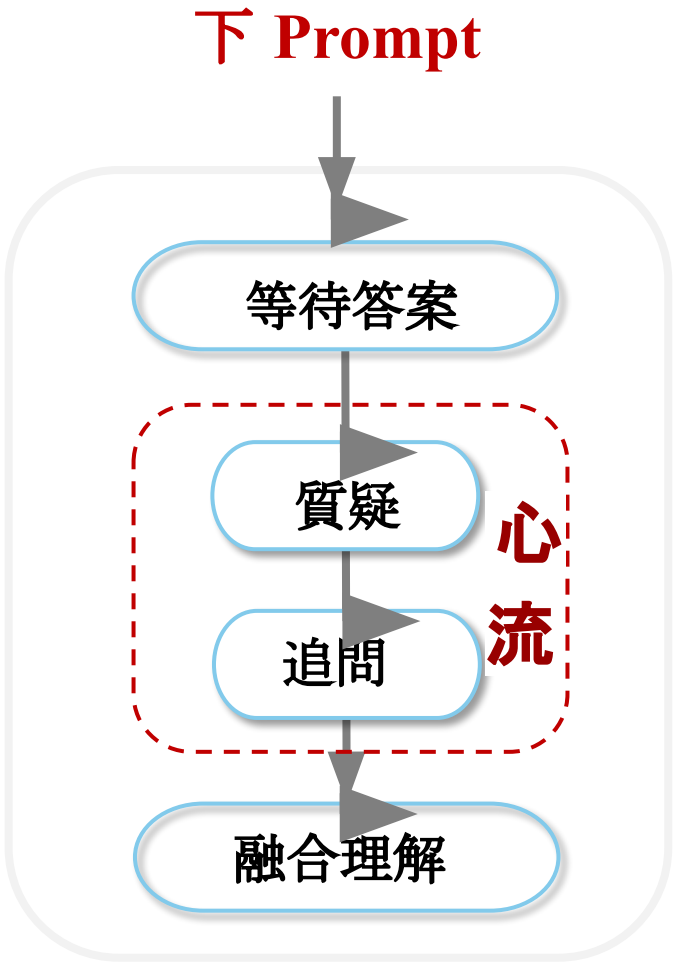
(因此 Merge 前最後做一次：Responsibility Check)

心流 (Flow)

- 米哈里·契克森米哈伊
  Mihaly Csikszentmihalyi

挑戰 ≈ 技能





首先要質疑

設置屏障

撤走屏障

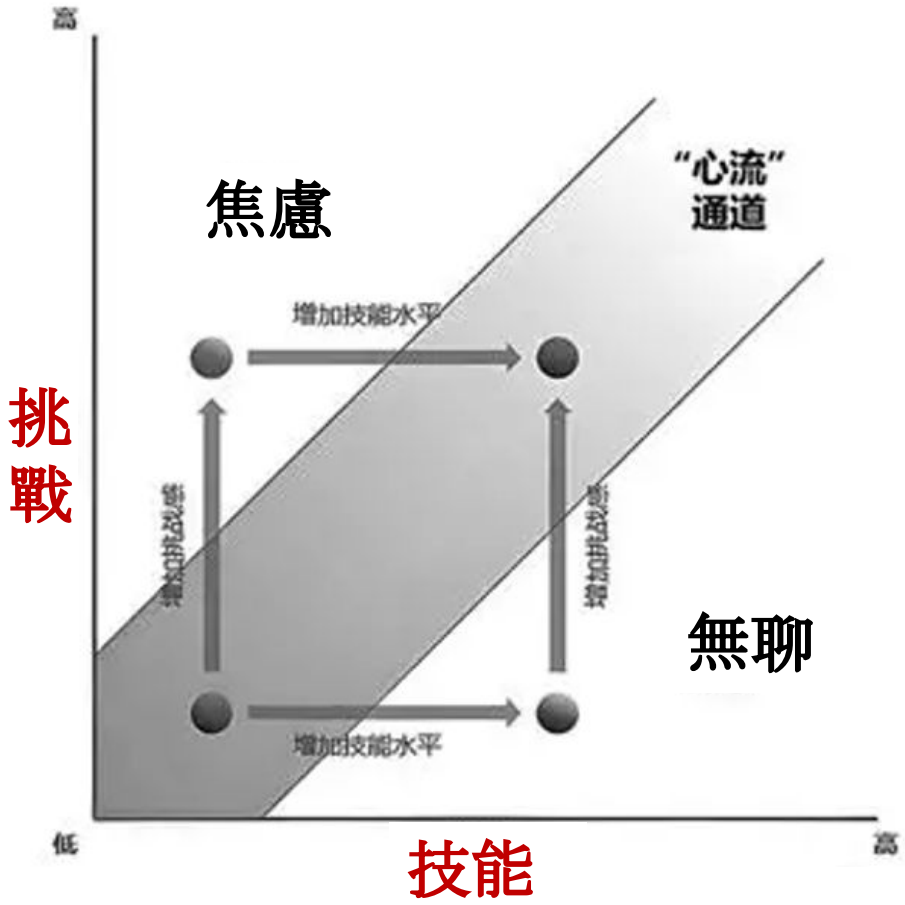
控制感

《心流》

心流的定義

心流(Flow)指人在進行某項活動時達到全神貫注、沉浸忘我的心理狀態，常伴隨時間感扭曲、行動流暢、內在愉悅與高度效率。

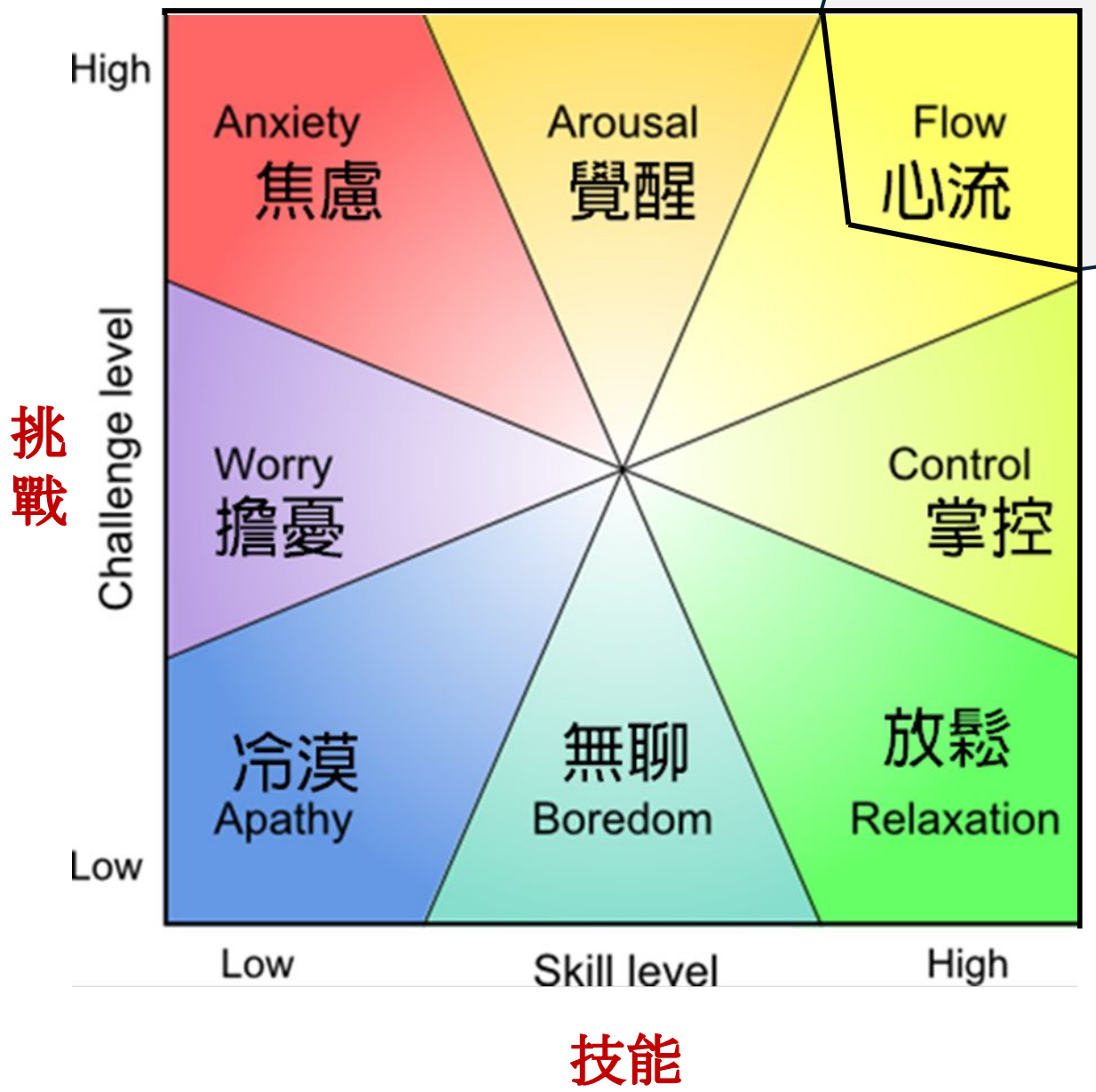
心流是一種高度專注、完全沉浸、忘我投入的最佳體驗狀態，其核心在於挑戰與技能的平衡、
明確目標與即時反饋，以及深度專注帶來的愉悅感。



米哈里·契克森米哈伊

開發者體驗的 幸福模型 FLOW

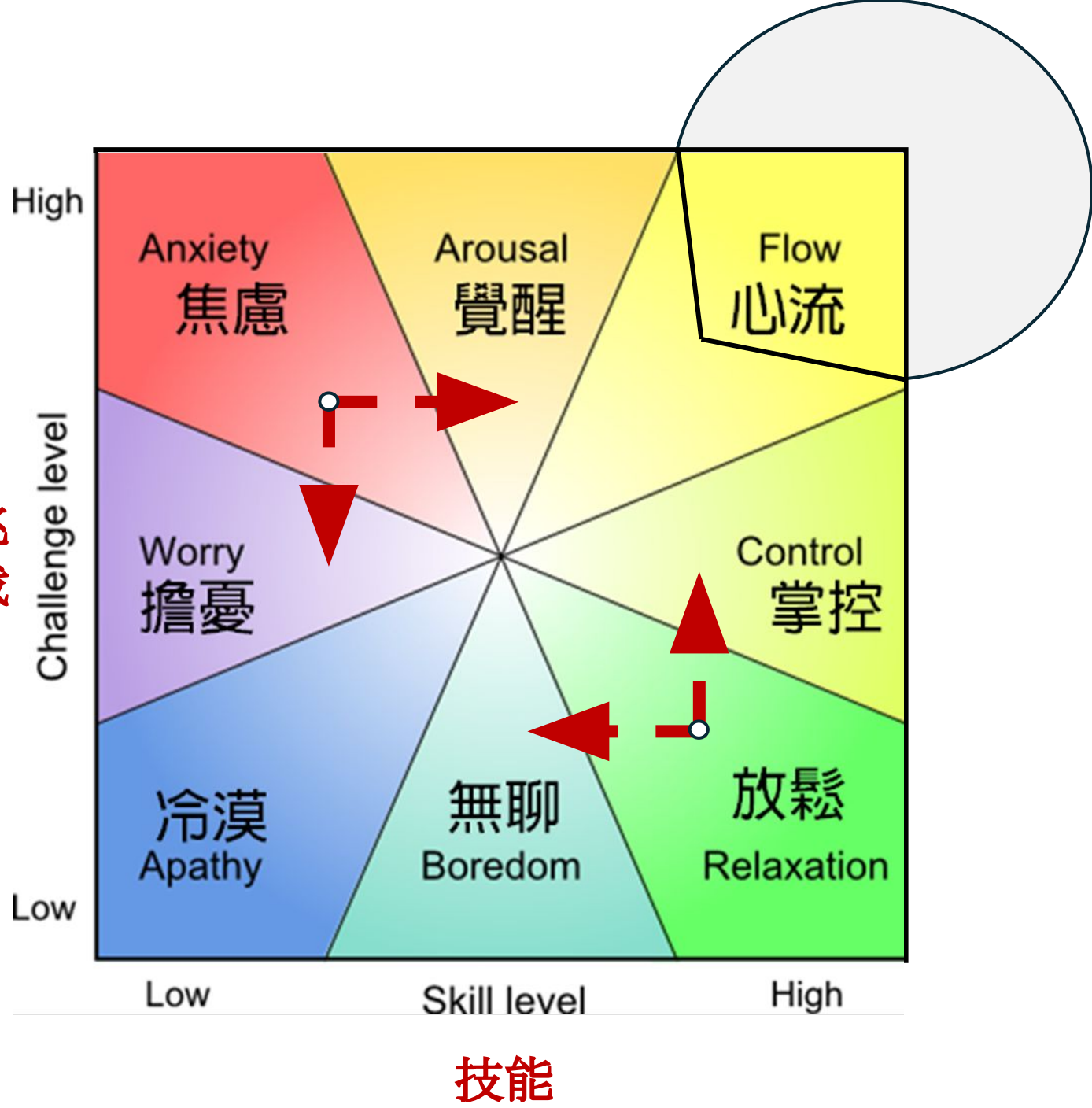
心流模型圖



當人們在**技能水準**和面臨的**挑戰難度**之間達到一定的平衡時，他們更有可能進入心流狀態。

開發者體驗的 幸福模型 FLOW

心流模型圖



- 關鍵點
  - 當你感到焦慮時…
  - 當你感到放鬆時…

回饋、反思

建立習慣/模式

需求拆解 與 工作拆解

- **需求拆解:解「為什麼、真正要什麼？」**
- **工作拆解:解「為了做到它，要做哪些事？」**

	需求拆解	工作拆解
核心問題	客戶真正想要什麼？	系統要做哪些事情？
對象	Business Intent	Feature / Function
關注	Why / Who / What	How / Component / Task
找的是	意圖、規則、假設、邊界	功能、責任、依賴、順序
主要產物	Requirement Model	Task Model
AI 的作用	**協助發現模糊與未知**	**協助拆成可執行 Task**
人的責任	確認意圖是否正確	確認拆法是否合理

提問

工作拆解四原則

四個提問，讓拆解有依據、可執行、可驗證



01 對成果

這件事讓哪個需求往前一步？

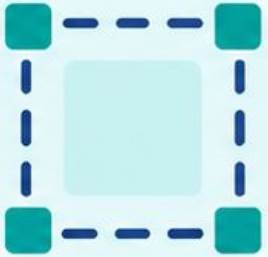
說清用途，連結需求



02 切邊界

負責到哪裡？怎樣算完成？

界定範圍，明確責任



03 查未知

哪些假設需要先確認？

找出未知，優先驗證



04 留證據

如何確認它做對了？

留下證據，共同檢查



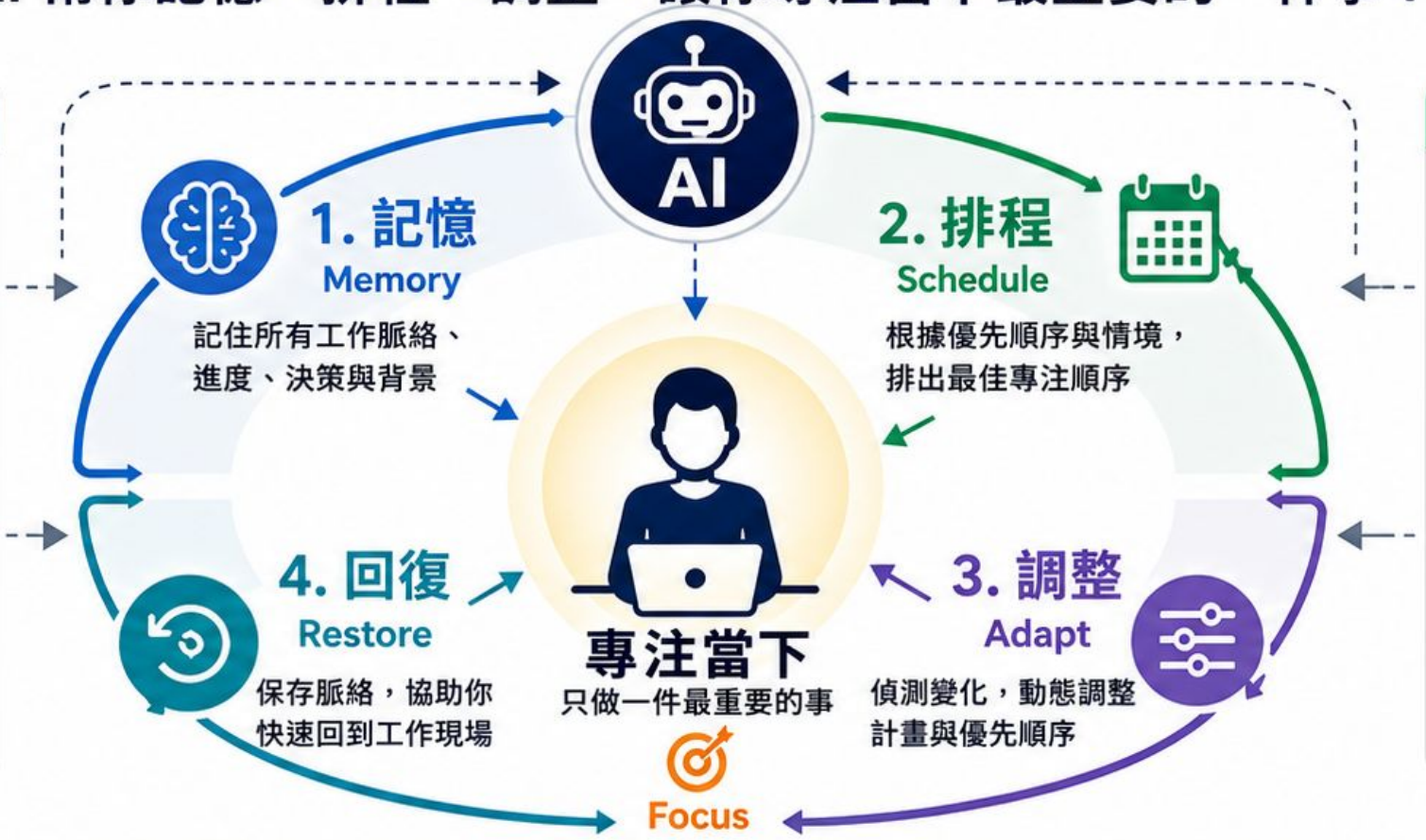
成果對得上 · 邊界說得清 · 未知先確認 · 完成有證據

動態專注框架 Dynamic Focus Framework

AI 幫你記憶、排程、調整，讓你專注當下最重要的一件事！

多工作業來源

- 專案開發 Project A
- 線上問題 Production Issue
- 團隊協作 Team Collaboration
- 程式審查 Code Review
- 等待回覆 Waiting
- 其他任務 Other Tasks



動態變化

- 緊急事件 Urgent Issue
- 需求變更 Requirement Change
- 他人回覆 Response Received
- 狀態更新 Status Update
- 其他變化 Other Changes

AI 產生的專注路徑 (你的課表)

開始 Start → 專案開發 Project A → 線上問題 Issue → 程式審查 Review → 專案開發 Project A → 團隊協作 Collaboration → ... → 結束 End

核心價值

讓 AI 管理複雜度，讓你保持專注與深度思考，
在快速變動中持續產出高品質的成果。

人：專注與決策　AI：記憶與調度

工作交給 AI，多工自然發生

拆開任務、委派執行，工程師便能推進下一項工作

拆開工作 → 委派 AI agent → 執行與注意力分離 → 多項工作同時進行

同一位工程師，三項進行中的任務



1. 委派後，轉向下一項任務
2. 結果回來，再接回脈絡
3. 多工來自非同步協作

AI 執行可以並行；人的注意力仍需逐項聚焦。

新的能力：安排切換，保存脈絡，驗證成果

工作 → 「封存」 → 切換 → 載入 → 恢復 → 工作

- **Where** ｜ 我在哪裡
- **Why** ｜ 我為什麼走到這裡
- **Think** ｜ 我現在怎麼想
- **Next** ｜ 我下一步準備驗證什麼
- **Resume** ｜ 回來第一個動作是什麼

只要開始第一個動作，後面的 Context 很可能就會自然恢復

AI 主動產生 Context Checkpoint

- **目前位置**
  API Timeout Root Cause Analysis
- **已排除**
  DB、Network
- **目前推理**
  問題集中於尖峰流量，
  因此懷疑 Connection Pool
- **當前假設**
  Production Pool Size 不足
- **下一步驗證**
  比較 Production / Staging 設定
- **重新開始點**
  打開 Production Config → 搜尋 Pool Size

AI 平行工作,人類單點專注

挑戰 ≈ 技能

AI 平行工作, 人類單點專注;離開時壓縮脈絡, 回來時恢復理解。

- 最好的動態多工, 不是讓人做更多事情, 而是讓人無論有多少事情, 都能保持思考的連續性。
- 我們要解決的是:當你不得不離開一件事情時, 如何讓你回來的時候, 不必重新開始。

